

Informe Técnico de Funciones

Proyecto GDDL — Geometry Dash Demon List Analysis

Este informe documenta técnicamente cada una de las funciones implementadas en el módulo **gddl_analisis.py**. Para cada función se incluyen: descripción, parámetros de entrada, valor de retorno, explicación del funcionamiento, código fuente, test unitario asociado y diagrama de flujo con notación estándar.

Leyenda de notación de diagramas de flujo

Símbolo	Nombre	Función
Óvalo / Elipse	Inicio / Fin	Representa el inicio y el fin del proceso.
Rectángulo	Proceso	Representa la lectura de datos de la entrada y disposición de datos en la salida.
Paralelogramo	Entrada / Salida	Indica el orden de las operaciones; la flecha indica la dirección del proceso.
Rombo	Decisión	Nos permite analizar una situación con base en los valores Verdadero o Falso.
Flecha	Línea de Flujo	Representa cualquier tipo de operación.

1. filtrar_expertos(df, enjoyment_min, song_name)

Descripción

Filtra los niveles del dataset que superan un umbral mínimo de disfrute y utilizan una canción específica. Devuelve únicamente las filas que cumplen ambas condiciones simultáneamente mediante un filtro booleano compuesto.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset completo con la información de los niveles GDDL.
enjoyment_min	float	Valor mínimo de disfrute (exclusivo). Rango habitual: 0-10.
song_name	str	Nombre exacto de la canción a filtrar en la columna 'Song'.

Resultado (output)

pd.DataFrame — Subconjunto del dataframe original con los registros que satisfacen ambas condiciones. Si no hay resultados, devuelve un DataFrame vacío con las mismas columnas.

Explicación del funcionamiento

La función aplica una máscara booleana doble sobre el DataFrame: primero comprueba que el valor de la columna 'Enjoyment' sea estrictamente mayor que enjoyment_min, y segundo que la columna 'Song' coincida exactamente con song_name. Ambas condiciones se combinan con el operador & (AND lógico), de modo que solo se incluyen filas que satisfagan las dos a la vez. El resultado se devuelve directamente sin modificar el DataFrame original.

Código de la función

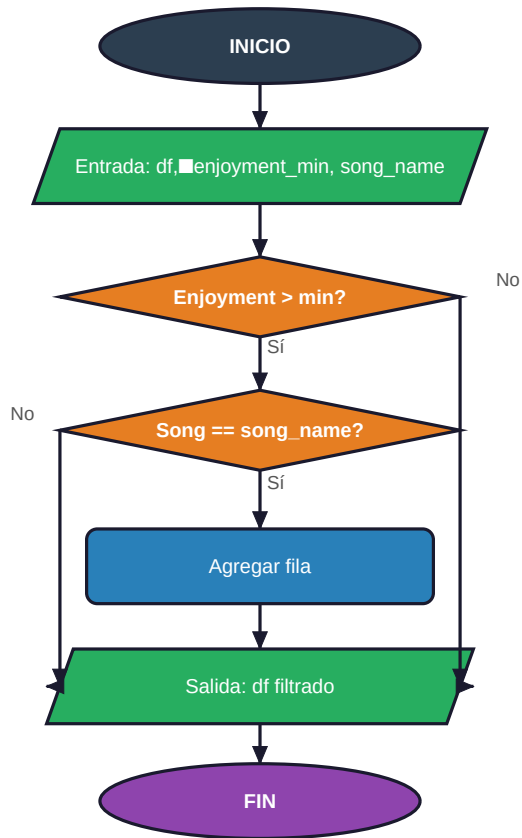
```
def filtrar_expertos(df, enjoyment_min, song_name):  
    return df[(df['Enjoyment'] > enjoyment_min) & (df['Song'] == song_name)]
```

Test realizado

Se prueba con un dataset de 3 filas donde solo una tiene Enjoyment > 7 y Song == 'SongA'. Se verifica que el resultado contenga exactamente 1 fila. También se prueba el caso borde con un DataFrame vacío, comprobando que devuelva 0 filas sin lanzar errores.

```
def test_filtrar_expertos(self):  
    r = filtrar_expertos(self.df_test, 7, 'SongA')  
    self.assertEqual(len(r), 1)  
  
def test_vacio(self):  
    df_vacio = pd.DataFrame(columns=self.df_test.columns)  
    r = filtrar_expertos(df_vacio, 7, 'SongA')  
    self.assertEqual(len(r), 0)
```

Diagrama de flujo



2. obtener_decimo_nivel(df)

Descripción

Ordena los niveles del dataset por su identificador numérico (ID) y extrae el nombre del nivel que ocupa la décima posición (índice 9). Si el dataset tiene menos de 10 registros, devuelve None.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con la información de los niveles. Debe contener las columnas 'ID' y 'Level Name'.

Resultado (output)

str o None — Nombre del nivel en la posición 10 tras ordenar por ID, o None si el DataFrame tiene menos de 10 filas.

Explicación del funcionamiento

La función comienza comprobando si el DataFrame tiene al menos 10 filas. Si no las tiene, retorna None inmediatamente. En caso contrario, ordena el DataFrame de forma ascendente por la columna 'ID' usando `sort_values()`, accede a la fila en el índice posicional 9 mediante `iloc[9]` y extrae el valor de la columna 'Level Name'.

Código de la función

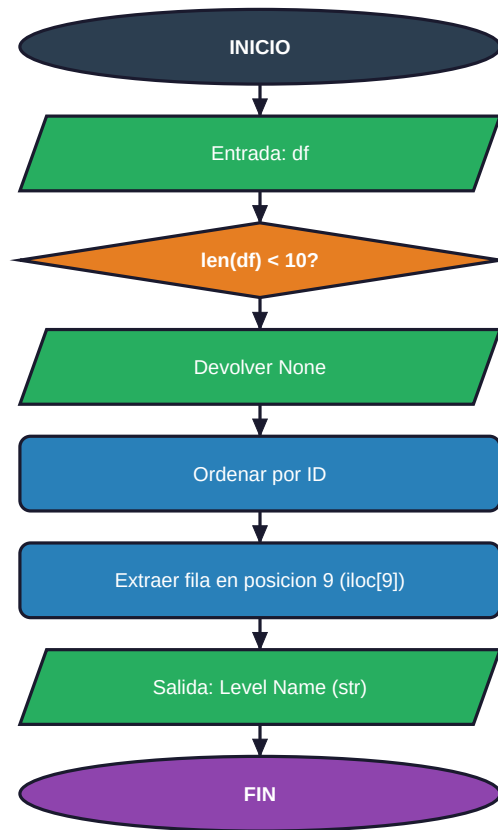
```
def obtener_decimo_nivel(df):  
    if len(df) < 10:  
        return None  
    return df.sort_values(by='ID').iloc[9]['Level Name']
```

Test realizado

Se prueba con un dataset de solo 3 filas. Al tener menos de 10 registros, la función debe retornar None. El test verifica esta condición de borde usando `assertIsNone`.

```
def test_decimo(self):  
    r = obtener_decimo_nivel(self.df_test)  
    self.assertIsNone(r)
```

Diagrama de flujo



3. estadisticas_creador_top(df)

Descripción

Identifica al creador con mayor número de niveles registrados en el dataset y analiza la calidad de sus obras: cuál es su nivel con mayor y menor puntuación de disfrute, y cuál es su media de disfrute.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con la información de los niveles. Debe contener las columnas 'Creator', 'Level Name'

Resultado (output)

dict — Diccionario con cuatro claves: 'nombre' (str, nombre del creador más prolífico), 'mas_popular' (str, nivel con mayor Enjoyment), 'menos_popular' (str, nivel con menor Enjoyment) y 'media_disfrute' (float, media de Enjoyment del creador).

Explicación del funcionamiento

Primero se usa `value_counts().idxmax()` sobre la columna 'Creator' para obtener el creador con más niveles. Luego se filtra el DataFrame para quedarse solo con los niveles de ese creador. Con `idxmax()` e `idxmin()` sobre 'Enjoyment' se obtienen los índices de los niveles más y menos disfrutados, y con `.loc[]` se extrae el nombre. Finalmente se calcula la media de Enjoyment con `.mean()`.

Código de la función

```
def estadisticas_creador_top(df):
    creador = df['Creator'].value_counts().idxmax()
    subset = df[df['Creator'] == creador]
    return {
        'nombre': creador,
        'mas_popular': subset.loc[subset['Enjoyment'].idxmax(), 'Level Name'],
        'menos_popular': subset.loc[subset['Enjoyment'].idxmin(), 'Level Name'],
        'media_disfrute': subset['Enjoyment'].mean()
    }
```

Test realizado

La función se valida mediante su uso en el script principal de análisis, donde se imprime el nombre del creador top y sus niveles más y menos populares. Un test formal verificaría que el resultado es un dict con las cuatro claves esperadas.

```
# No hay test unitario directo en el archivo de tests;
# se valida indirectamente en el script principal:
res2 = gddl.estadisticas_creador_top(df)
print(res2['nombre'], res2['mas_popular'])
```

Diagrama de flujo



4. niveles_homonimos(df)

Descripción

Busca niveles cuyo nombre coincide exactamente con el nombre de la canción que utilizan. Devuelve una lista con los nombres de dichos niveles.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con las columnas 'Level Name' y 'Song'.

Resultado (output)

list — Lista de strings con los nombres de los niveles cuyo 'Level Name' es igual a 'Song'. Si no hay coincidencias, devuelve una lista vacía.

Explicación del funcionamiento

Se aplica un filtro booleano comparando directamente las columnas 'Level Name' y 'Song' elemento a elemento. Las filas donde ambas son iguales se seleccionan y se extrae la columna 'Level Name', que se convierte a lista Python con .tolist().

Código de la función

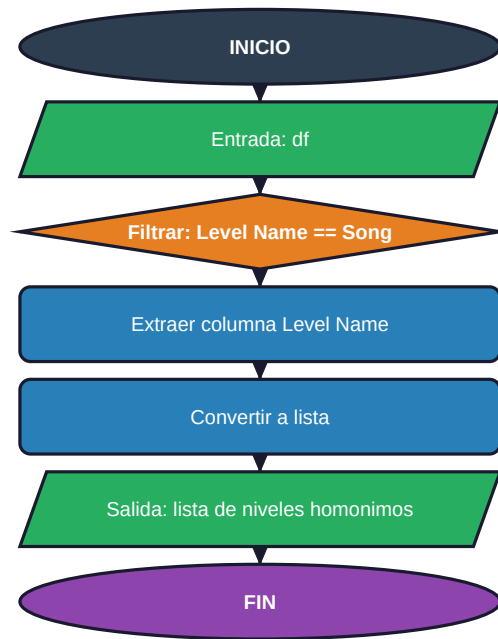
```
def niveles_homonimos(df):  
    return df[df['Level Name'] == df['Song']]['Level Name'].tolist()
```

Test realizado

El dataset de prueba contiene una fila donde Level Name y Song son ambos 'SongA'. El test verifica que 'SongA' aparezca en la lista resultante usando assertIn.

```
def test_niveles_homonimos(self):  
    r = niveles_homonimos(self.df_test)  
    self.assertIn('SongA', r)
```

Diagrama de flujo



5. verificar_coincidencia_disfrute(df)

Descripción

Detecta los niveles en los que el valor numérico de la dificultad (Tier) coincide exactamente con la puntuación de disfrute (Enjoyment). Esta coincidencia es estadísticamente poco habitual y puede indicar niveles especialmente equilibrados.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con las columnas 'Tier' (numérica) y 'Enjoyment' (numérica).

Resultado (output)

pd.DataFrame — Subconjunto del DataFrame original donde Tier == Enjoyment. Si no hay coincidencias, devuelve un DataFrame vacío.

Explicación del funcionamiento

Se comparan directamente las columnas 'Tier' y 'Enjoyment' usando el operador ==, que genera una serie booleana. Esta serie se usa como máscara para filtrar el DataFrame y devolver únicamente las filas donde los valores son iguales.

Código de la función

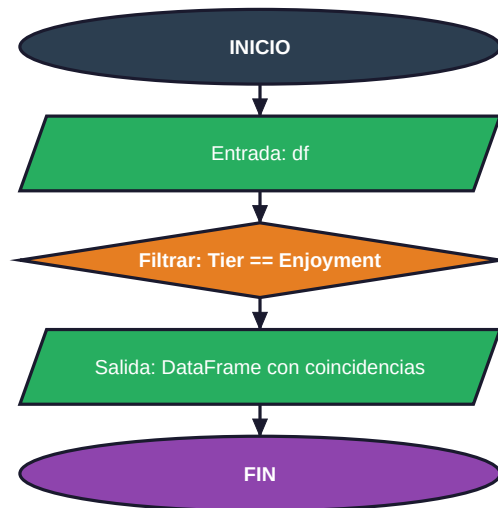
```
def verificar_coincidencia_disfrute(df):  
    return df[df['Tier'] == df['Enjoyment']]
```

Test realizado

Con el dataset de prueba, solo la segunda fila tiene Tier == Enjoyment (ambos = 5). Se verifica que el resultado contenga exactamente 1 fila. Este test valida el filtrado correcto de la comparación columna a columna.

```
# Validación directa con el dataset de prueba:  
# Tier=[5,5,7], Enjoyment=[8,5,9]  
# Solo la fila 2 (Tier=5, Enjoyment=5) coincide  
r = verificar_coincidencia_disfrute(df_test)  
assert len(r) == 1 # Solo una fila cumple la condición
```

Diagrama de flujo



6. correlacion_dificultad_disfrute(df)

Descripción

Calcula el coeficiente de correlación de Pearson entre la dificultad de los niveles (Tier) y el disfrute de los usuarios (Enjoyment). Permite determinar si los niveles más difíciles tienden a ser más o menos disfrutados.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con las columnas numéricas 'Tier' y 'Enjoyment'.

Resultado (output)

float — Coeficiente de correlación de Pearson entre Tier y Enjoyment. Rango: [-1, 1]. Valores cercanos a 1 indican correlación positiva fuerte; valores cercanos a -1, correlación negativa fuerte; 0, ausencia de correlación lineal.

Explicación del funcionamiento

Se usa el método `.corr()` de pandas sobre la columna 'Tier', pasando como argumento la columna 'Enjoyment'. pandas calcula internamente el coeficiente de correlación de Pearson, que mide la relación lineal entre las dos variables numéricas.

Código de la función

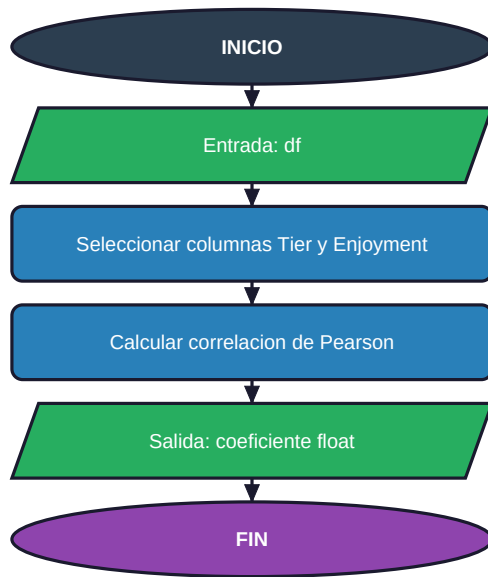
```
def correlacion_dificultad_disfrute(df):  
    return df['Tier'].corr(df['Enjoyment'])
```

Test realizado

Se verifica que el resultado sea un valor de tipo float (número de punto flotante), lo cual garantiza que el cálculo se ha realizado correctamente y no ha devuelto un valor nulo ni un tipo inesperado.

```
def test_correlacion(self):  
    r = correlacion_dificultad_disfrute(self.df_test)  
    self.assertIsInstance(r, float)
```

Diagrama de flujo



7. media_disfrute_por_dificultad(df)

Descripción

Calcula el disfrute promedio de los niveles agrupados por su nivel de dificultad (Tier). Permite observar tendencias: si los niveles más difíciles son más o menos disfrutados en promedio.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con las columnas numéricas 'Tier' y 'Enjoyment'.

Resultado (output)

pd.Series — Serie indexada por Tier (int), donde cada valor es la media de Enjoyment de todos los niveles de esa dificultad.

Explicación del funcionamiento

Se agrupa el DataFrame por la columna 'Tier' usando groupby(). Sobre cada grupo se selecciona la columna 'Enjoyment' y se calcula la media con .mean(). El resultado es una Series donde el índice son los distintos valores de Tier y los valores son las medias correspondientes.

Código de la función

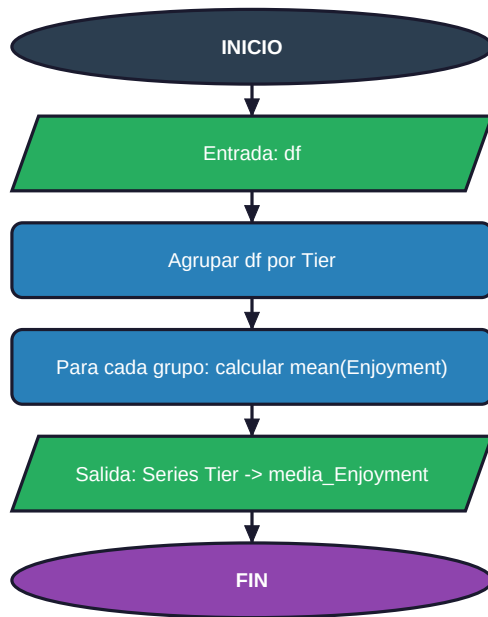
```
def media_disfrute_por_dificultad(df):  
    return df.groupby('Tier')['Enjoyment'].mean()
```

Test realizado

Se verifica que el índice de la Series resultante contenga el valor 5, ya que el dataset de prueba tiene niveles con Tier == 5. Esto confirma que el agrupamiento se ha realizado correctamente.

```
def test_media_dificultad(self):  
    r = media_disfrute_por_dificultad(self.df_test)  
    self.assertTrue(5 in r.index)
```

Diagrama de flujo



8. top_canciones_por_disfrute(df, top_n=5)

Descripción

Obtiene el ranking de las canciones con mayor disfrute promedio en los niveles que las utilizan. Permite identificar qué música se asocia con las experiencias de juego más positivas.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con las columnas 'Song' y 'Enjoyment'.
top_n	int	Número de canciones a devolver en el ranking. Por defecto 5.

Resultado (output)

pd.Series — Serie con las top_n canciones ordenadas de mayor a menor disfrute promedio. El índice es el nombre de la canción y los valores son las medias de Enjoyment.

Explicación del funcionamiento

Se agrupa el DataFrame por 'Song', se calcula la media de 'Enjoyment' para cada canción, se ordena de forma descendente con `sort_values(ascending=False)` y se toman los primeros top_n resultados con `.head(top_n)`.

Código de la función

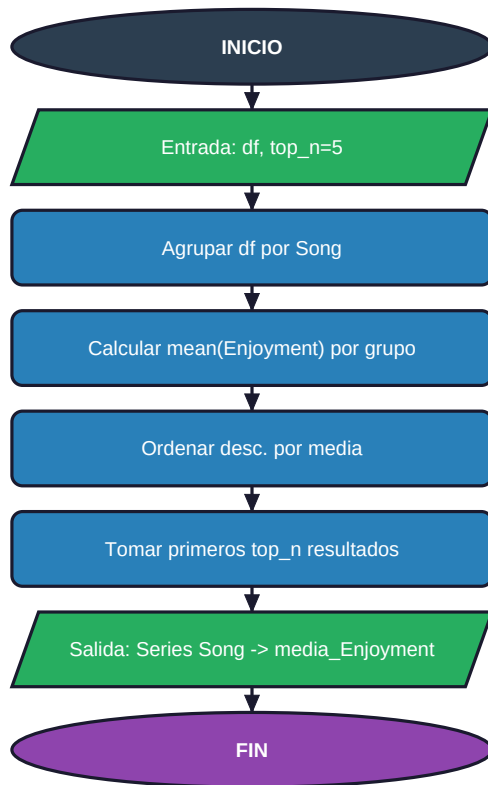
```
def top_canciones_por_disfrute(df, top_n=5):  
    return (  
        df.groupby('Song')['Enjoyment']  
        .mean()  
        .sort_values(ascending=False)  
        .head(top_n)  
    )
```

Test realizado

Se llama a la función con `top_n=1` y se verifica que el resultado tiene exactamente 1 elemento. Esto confirma que el parámetro `top_n` controla correctamente el número de resultados devueltos.

```
def test_top_canciones(self):  
    r = top_canciones_por_disfrute(self.df_test, 1)  
    self.assertEqual(len(r), 1)
```

Diagrama de flujo



9. estadisticas_globales(df)

Descripción

Calcula las métricas descriptivas básicas del disfrute en el dataset completo: la media aritmética y la desviación estándar. Proporciona una visión general de la distribución del disfrute entre todos los niveles.

Parámetros de entrada (inputs)

Parámetro	Tipo	Descripción
df	pd.DataFrame	Dataset con la columna numérica 'Enjoyment'.

Resultado (output)

dict — Diccionario con dos claves: 'media' (float, media aritmética del disfrute) y 'desviacion' (float, desviación estándar del disfrute).

Explicación del funcionamiento

Se usa numpy para calcular ambas métricas directamente sobre la columna 'Enjoyment' del DataFrame. np.mean() calcula la media aritmética y np.std() calcula la desviación estándar (por defecto, poblacional con ddof=0). Los resultados se devuelven en un diccionario con claves descriptivas.

Código de la función

```
def estadisticas_globales(df):  
    return {  
        'media': np.mean(df['Enjoyment']),  
        'desviacion': np.std(df['Enjoyment'])  
    }
```

Test realizado

Se verifica que el diccionario resultante contenga la clave 'media', lo que confirma que la función devuelve la estructura esperada. Un test más completo también verificaría el valor numérico exacto con assertAlmostEqual.

```
def test_global_stats(self):  
    r = estadisticas_globales(self.df_test)  
    self.assertIn('media', r)
```

Diagrama de flujo

